

2do Parcial - Técnica y Diseño de Algoritmos (ex Algo3)

Simulacro #1 - 2do Cuatrimestre 2025

Nombre y Apellido	#Orden	L.U.	Turno	Corrector	Nota

	Ej 1	Ej 2	Ej 3	Ej 4
Puntajes parciales				

Duración: 4 horas. Este examen es individual y a libro cerrado. La nota de aprobación es de 60/100 puntos.

Ej 1: (16 Puntos) Marque la única opción correcta.

Puntajes: Correcto: +4 pts | Incorrecto: -2 pt | En blanco: 0 pt.

I. Sea un grafo simple no dirigido $G=(V,E)$ con $|V|=n$, $|E|=m$. Se lo representa con listas de adyacencia (arreglos de listas). ¿Cuál de las siguientes afirmaciones es correcta (en el peor caso)?

- A) Verificar si existe la arista $\{u,v\}$ se puede hacer en $O(1)$.
- B) Iterar todos los vecinos de un vértice u cuesta $O(\deg(u))$.
- C) Iterar todas las aristas cuesta $O(n)$.
- D) Agregar una arista $\{u,v\}$ cuesta $O(\deg(u)+\deg(v))$ necesariamente.

II. Sea G un digrafo. Al correr DFS desde un vértice s , se obtiene un árbol DFS T y la clasificación usual de aristas (tree/forward/back/cross). ¿Cuál es verdadera?

- A) Si el DFS encuentra una back-edge, entonces G es fuertemente conexo.
- B) Si G es acíclico, entonces en cualquier DFS no existen back-edges.
- C) Si en un DFS no aparecen cross-edges, entonces G es un árbol.
- D) En un digrafo, la existencia de una forward-edge implica la existencia de un ciclo.

III. Sea $G=(V,E)$ un grafo conexo no dirigido con pesos reales. Sea e en E . ¿Cuál condición es suficiente para asegurar que e pertenece a todo Árbol Generador Mínimo (AGM) de G ?

- A) e es la arista de menor peso del grafo.
- B) e es la arista de menor peso que cruza algún corte $(S, V \setminus S)$ y es única con ese mínimo en ese corte.
- C) e pertenece a algún ciclo simple de G y no es la de mayor peso del ciclo.
- D) Todos los pesos del grafo son distintos.

IV. Sea $G=(V,E)$ un digrafo con pesos (posiblemente negativos). Queremos calcular distancias mínimas desde una fuente s . ¿Cuál afirmación es correcta?

- A) Dijkstra funciona siempre que no haya ciclos negativos.
- B) Bellman-Ford detecta ciclos negativos alcanzables desde s y, si no hay, obtiene distancias mínimas.
- C) Floyd-Warshall solo sirve para grafos no dirigidos.
- D) Si hay una arista de peso negativo, entonces necesariamente existe un ciclo negativo.

(Tema/s que evalúa: representación y complejidades; DFS/BFS y propiedades; AGM por corte/ciclo; caminos mínimos y algoritmos.)

Ej 2: (28 Puntos) Grafos sin pesos (DFS/BFS).

Sea $G=(V,E)$ un grafo no dirigido, simple y conexo, con $|V|=n$, $|E|=m$. Se corre un DFS desde una raíz r , obteniendo el árbol DFS T . Para cada vértice v , sea $\text{tin}[v]$ su tiempo de descubrimiento (preorden). Definimos $\text{low}[v]$ como el menor tin alcanzable desde v usando cero o más aristas de árbol hacia abajo y a lo sumo una back-edge hacia arriba.

(a) (18 pts) Demostración teórica

Demostrar que:

- La raíz r es punto de articulación si y solo si tiene al menos dos hijos en T .
- Un vértice $v \neq r$ es punto de articulación si y solo si existe un hijo u de v en T tal que $\text{low}[u] \geq \text{tin}[v]$.

Se pide una demostración clara, usando componentes conexas en $G - v$ y el significado de low .

(b) (10 pts) Algoritmo

Diseñar un algoritmo que, dado G , compute todos los puntos de articulación. Debe:

- Usar la propiedad del inciso (a) como caja negra (o re-demostrarla muy brevemente).
- Describir qué se computa durante el DFS (al menos tin , low y cantidad de hijos de la raíz).
- Justificar que corre en $O(n+m)$.

(Tema/s que evalúa: DFS, back-edges, low-link; conectividad y puntos de articulación; diseño y análisis $O(n+m)$.)

Ej 3: (28 Puntos) DAG + DP (modelado y caminos en DAG).

En una empresa, un proyecto se divide en n tareas $v_1, \dots, v_n$. Cada tarea v_i tiene una duración $t_i \geq 0$. Para comenzar v_i , deben estar terminadas todas las tareas de un conjunto $N(v_i) \subset \{v_1, \dots, v_{i-1}\}$ (siempre se depende de tareas con índice menor, por lo que no hay ciclos). Se pueden ejecutar tareas en paralelo sin límite de recursos.

Definimos la duración total del proyecto como el menor tiempo T tal que existe un plan de ejecución (respetando dependencias) que termina todas las tareas a tiempo T .

Una tarea v_i se dice crítica si cualquier aumento (por pequeño que sea) en t_i necesariamente aumenta la duración total T .

(a) (4 pts) Verdadero/Falso con justificación

“Si una tarea no es crítica, entonces reducir su duración nunca puede reducir la duración total del proyecto.” Decidir si es verdadero o falso y justificar (se acepta contraejemplo).

(b) (14 pts) Modelado + algoritmo

Modelar el problema con un grafo dirigido acíclico $G=(V,E)$ y diseñar un algoritmo que calcule:

- 1) la duración total T , y
- 2) el conjunto de tareas críticas.

Debe usarse como caja negra algoritmos estándar sobre DAG (por ejemplo: orden topológico + DP). Aclarar qué valores se calculan (por ejemplo: tiempo más temprano de inicio/fin y tiempo más tardío de inicio/fin u otra formulación equivalente).

(c) (5 pts) Corrección

Justificar por qué el algoritmo del inciso (b) devuelve el T correcto y caracteriza correctamente cuáles tareas son críticas.

(d) (5 pts) Complejidad

Analizar complejidad temporal y espacial en función de n y de $\sum_i |N(v_i)|$ (se espera algo lineal en el tamaño del grafo).

(Tema/s que evalúa: DAG, orden topológico, DP en DAG (camino más largo/tiempos), modelado de dependencias, corrección y complejidad.)

Ej 4: (28 Puntos) Flujo en redes (modelado).

En un torneo por rondas con n equipos, al final de la ronda se conoce el puntaje crudo p_i de cada equipo i . Para decidir quién pasa a la siguiente ronda, el puntaje de clasificación se satura en P : el puntaje que cuenta es $c_i = \min(p_i, P)$. Si $p_i > P$, el excedente $s_i = p_i - P$ se considera puntos sobrantes que el equipo i puede regalar.

Se conoce también una lista de resultados de la ronda: para cada partido jugado, se sabe quién le ganó a quién. Modelamos esto como un digrafo R donde hay una arista $i \rightarrow j$ si el equipo i le ganó al equipo j (solo en ese caso i puede regalarle puntos a j).

Reglas de redistribución:

- 1) Cada equipo i puede regalar en total a lo sumo s_i puntos.
- 2) Un equipo j puede recibir en total a lo sumo Q puntos (límite anti-desbalance).
- 3) Los puntos regalados sí aumentan el puntaje de clasificación del receptor, pero también se satura: el puntaje final es $c'_j = \min(c_j + \text{recibido}_j, P)$.

En esta ronda, solo queda eliminado el equipo con menor puntaje final (los otros $n-1$ pasan). El equipo “-Echu” tiene $c_E = K$ con $K < P$, y no tiene puntos sobrantes ($s_E = 0$). Los demás equipos se quieren poner de acuerdo para redistribuir puntos sobrantes con el objetivo de que -Echu quede último, o sea, que al finalizar la redistribución se cumpla: para todo $i \neq E$, $c'_i \geq K+1$.

(a) (10 pts) Modelado como flujo

Construir una red de flujo (nodos, aristas y capacidades) que permita decidir si existe una redistribución válida que logre que -Echu quede eliminado (i.e., que todos los demás terminen con al menos $K+1$). Aclarar exactamente qué condición de flujo máximo/valor del flujo se chequea para responder SI/NO.

(b) (8 pts) Justificación de corrección

Justificar por qué:

- Toda redistribución válida induce un flujo factible en tu red, y
- Todo flujo factible que cumple tu condición induce una redistribución válida.

(c) (5 pts) Complejidad

Dar el tamaño de la red en función de n y $|E(R)|$, y analizar la complejidad usando, por ejemplo, Edmonds-Karp (o el algoritmo de flujo que prefieras, justificando).

(d) (5 pts) Variante

Ahora además se impone que para cada arista $i \rightarrow j$ del digrafo R existe un límite U_{ij} y el equipo i puede regalarle a j a lo sumo U_{ij} puntos (además de las reglas anteriores). Explicar cómo modificar tu red para contemplar esta restricción.

(Tema/s que evalúa: modelado con flujo, factibilidad por capacidades, construcción de red, corrección del modelo, complejidad (tamaño + EK).)